Event Tracing for Windows (ETW) Security: Attack Techniques and Defense Strategies

Prepared by: Birkan KES
Date: 25/08/2025

İçindekiler

GİRİŞ 2
ETW'nin Tanıtımı 3
ETW Architecture 4
ETW Altyapısının Teknik Detayları 5
IBM'in ETW threat modeli 6
SALDIRI CEŞİTLERİ 7
ETW Tampering 7
Çevresel/Ortamsal (Env/Registry) Saldırılar 9
User Mode ETW Sağlayıcılarına Yönelik Saldırılar 10
Çekirdek Modu ETW Sağlayıcılarına Yönelik Saldırılar 11
ETW Güvenligi Savunma Stratijileri ve önerileri 12
Referanslar 15

GİRİŞ

Bu yazıya, aklımda kurduğum bir saldırı teorisini denemek amacıyla yazmaya başladım. Bu süreçte edindiğim bilgileri ve karşılaştığım saldırı tekniklerini paylaşmak istedim.

Siber savunma dünyasında "görünürlük", olayların doğru şekilde kaydedilmesi ve tehditlerin zamanında tespit edilmesi için en kritik adımdır. İşte bu noktada Windows'un Event Tracing for Windows (ETW) altyapısı devreye girer. ETW, sistemin hem kullanıcı hem de çekirdek modundan gelen olaylarını kaydederek EDR, SIEM ve antivirüs çözümlerine veri sağlayan en önemli telemetri mekanizmalarından biridir.

Geçmişte ETW manipülasyonu yalnızca APT gruplarının başvurduğu sofistike bir yöntem olarak görülüyordu. Ancak bu algı değişti. Günümüzde ETW API patching, syscall manipülasyonu, kernel tabanlı DKOM saldırıları ve log dosyalarının doğrudan silinmesi gibi yöntemlerin, yalnızca gerçek saldırılarda değil test senaryolarında da kullanıldığını görüyoruz. Üstelik artık yalnızca APT grupları değil, fidye yazılımı çeteleri de ETW'yi hedef alıyor.

Bu yazıda ETW'nin yapısını, saldırganların kullandığı atlatma yöntemlerini ve savunma stratejilerini inceleyeceğiz. Edindiğimiz bilgiler doğrultusunda ise kendi önerilerimizi ve stratejimizi geliştirmeye çalışacağız.

ETW’nin Tanıtımı

ETW (Event Tracing for Windows), Windows işletim sisteminde çekirdek seviyesinde gömülü bir izleme altyapısıdır. Hem kullanıcı modu uygulamalarından hem de çekirdek mod sürücülerinden gelen olayları (events) kaydetmeyi sağlar. Birçok EDR ürünü ve güvenlik aracı ETW üzerinden telemetri topladığı için, saldırganlar ETW’yi atlatmaya veya bozarak algılamayı engellemeye çalışır.

ETW mimarisinin dört temel bileşeni vardır: Olay Sağlayıcısı (Provider), Oturum izleyicisi (Session), Kontroller (Controller) ve Tüketici (Consumer)

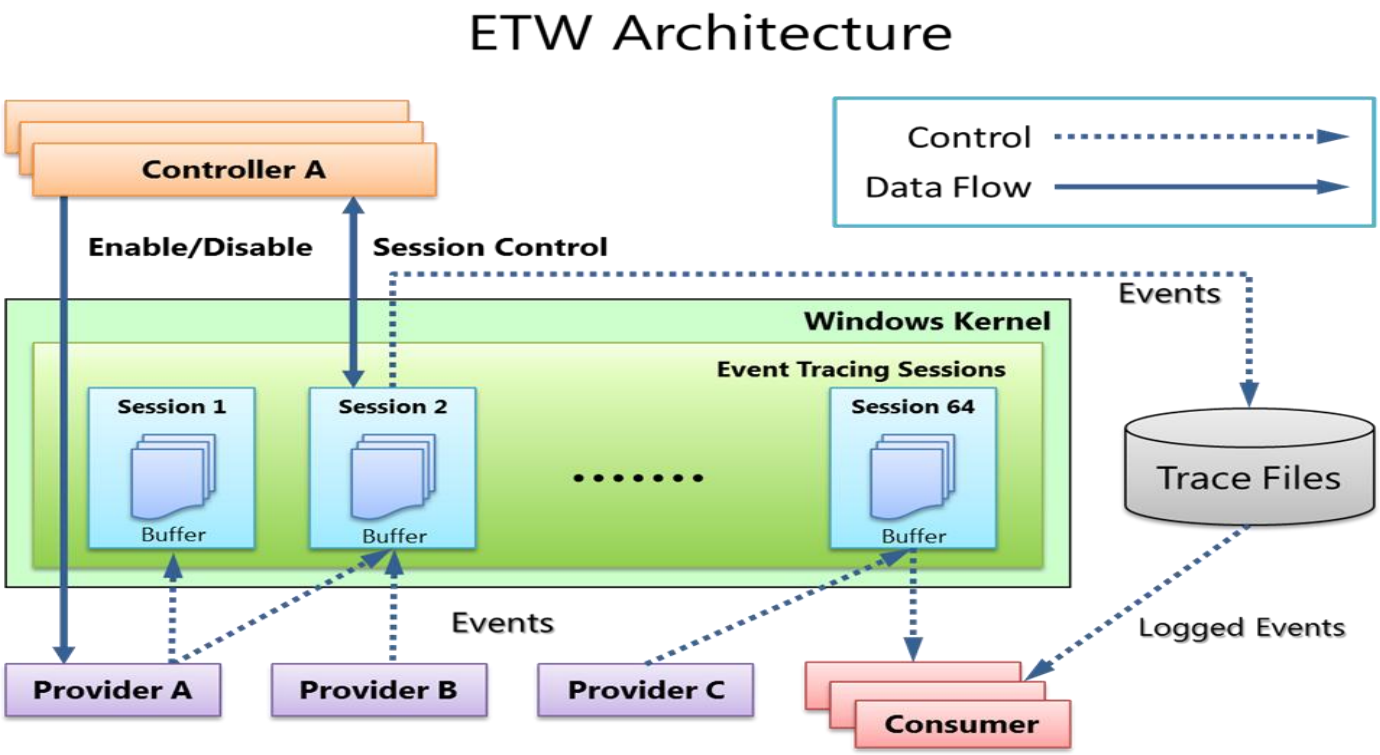
- **Provider:** Olay üreten izleme için yapılandırılmış bileşenlerdir. Kullanıcı seviyesi ya da çekirdeki seviyesi bir uygulama veya bizzat çekirdek sürücü Provider olabilir. Sağlayıcılar, ETW’ye kayıt yaptırarak, kod içindeki belirli noktaları işaretleyip olay verisi gönderirler. Her provider, bir XML manifest dosyasında bir GUID ile tanımlanır. Manifest’te ayrıca olay şablonları, görevler ve tanımlı veriler yer alır.

- **Session:** Bir kernel nesnesi olarak çalışır. Bir veya daha fazla provider’dan gelen olayları çekirdek bellek alanlarında toplar ve bunları disk üzerine .etl dosyaları olarak yazar veya gerçek zamanlı olarak consumer’lara iletir. Kullanıcı, birden fazla provider’ı tek bir oturuma eşleyebilir; bu sayede farklı kaynaklardan gelen olaylar aynı oturumda toplanabilir. Windows çekirdeği aynı anda en fazla 64 izleme oturumu destekler; bunların 8’i sistemsel (örn. NT Kernel Logger, Global Logger) adlı ön tanımlı oturumlardır. Özellikle NT Kernel Logger adlı oturum, çekirdek düzeyindeki olayları (işlem/iş parçacığı başlatma, DPC, kesme vb.) kaydetmek için ayrılmıştır ve sistemde yalnızca bir tane aktif olabilir

- **Controller:** ETW oturumlarını başlatan, durduran ve yapılandıran bileşendir. Session’ları izlemek/takip etmek icin kullanılan bir bileşendir.Controller, hangi provider’ların etkinleştirileceğine karar verir ve oturum parametrelerini belirler. Örneğin Windows komut satırındaki logman veya Windows Performance Recorder (WPR) gibi araçlar Controller görevi görür. Bir Controller çalıştırıldığında (EnableTrace veya StartTrace fonksiyonuyla), ilgili oturum oluşturulur ve istendiğinde durdurulur. İstenirse Windows kendi controller’larınızı yazmanıza olanak tanır.

- **Consumer:** Toplanan olayları okuyan ve işleyen uygulamalardır. Bir consumer, ya oluşturulmuş .etl günlük dosyasını okur, ya da aktif bir izleme oturumuna gerçek zamanlı olarak bağlanır. Örneğin Event Viewer, Windows Performance Analyzer (WPA) veya özel analiz araçları ETW Consumer olabilir. İşletim sistemindeki Resource Monitor gibi bileşenler de ETW tüketici örneklerindendir.

ETW Architecture



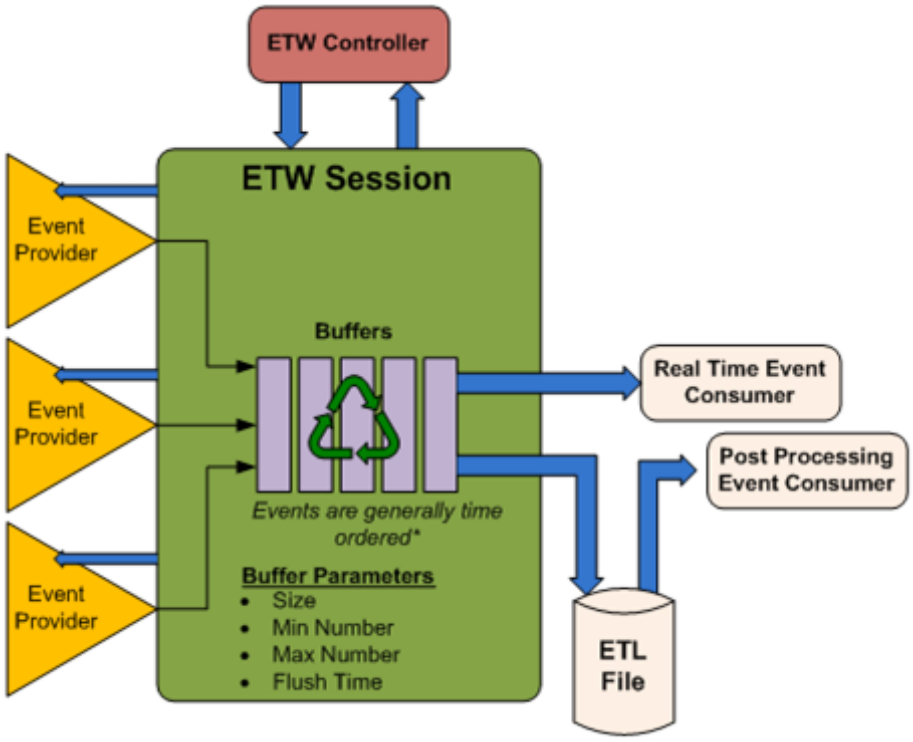
REF: https://learn.microsoft.com/en-us/windows-hardware/test/weg/images/weg-diagram-etw-architecture-four-components.png

Controllers Provider'ları etkinleştirir, Event Tracing Sessions olayları Buffer'larda toplar ve bunları dosyaya veya tüketicilere iletir.

ETW'de olay akışı şu şekilde gerçekleşir:

1. **Oturum Başlatma**: Controller yeni bir session oluşturur ve ilgili provider'ları etkinleştirir. Bu aşamada oturumun adı, hedef .etl dosyası ve kaydedilecek olay türleri belirlenir.
2. **Olay Üretimi**: Etkinleşen provider'lar, kodlarında ETW API çağrılarıyla (EventWrite , TraceLoggingWrite vb.) Event gönderirler.
3. **Olay Toplama**: Çekirdek, gelen olayları kernel modundaki bellek alanlarına yazar. Bunlar önceden ayrılmış sabit boyutlu alanlardır. Veri sabit alanlarda tutulur (circular buffer) ya da (alan dolunca) disk üzerine aktarılır.
4. **Veri İletimi**: Toplanan olay verileri iki şekilde kullanılabilir. Birincisi gerçek zamanlı iletimdir: Kernel, olayları doğrudan bağlı olan consumer süreçlerine yollar (WPA, Event Viewer vb.). İkincisi, günlük tabanlı (log-based)dır: Olaylar öncelikle diskte bir .etl dosyasına yazılır; analiz için daha sonra bu dosya okunur.
5. **Kullanım ve Analiz**: Consumer uygulamalar, .etl dosyalarını parse eder ya da anlık bağlantılar üzerinden gelen olay akışını işler. Örneğin Event Viewer, ETL'yi EVTX formatına çevirir; Windows Performance Analyzer ise performans sayaçlarını görselleştirir
6. **Gerçek zamanlı vs Günlük (log-based) kayıt**: Gerçek zamanlı mod, düşük gecikmeli izleme gerektiren senaryolarda kullanılır (örn. canlı performans takibi). Günlük kaydı ise olayları kalıcı tutar ve offline analiz olanağı sağlar. Kernel'da bufferlarda biriken olaylar belli büyüklüğe ulaştığında otomatik .etl dosyasına yazılır.

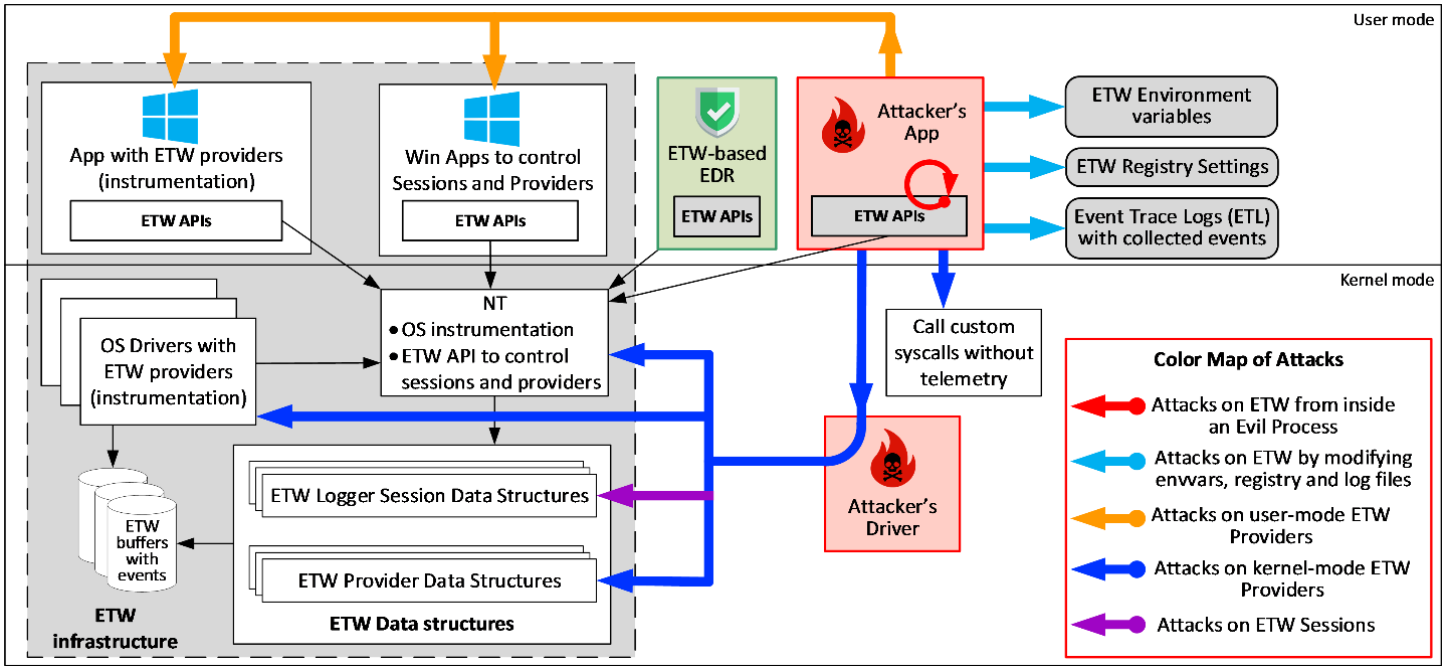
ETW Altyapısının Teknik Detayları



REF:https://learn.microsoft.com/en-us/previous-versions/windows/desktop/xperf/images/image070.png

1. **Oturum (Trace Session):** Her izleme oturumu kayıtlı bir isimle tanımlanır ve 64 bitlik bir GUID ile yönetilir. Windows varsayılan olarak eşzamanlı 64 oturuma izin verir; bunlardan 8'i sistem seviyesindeki ön tanımlı oturumlardır (ör. NT Kernel Logger, Global Logger, Circular Kernel Logger) . Kullanıcı modunda kendi oturumları (Private Logger) veya sisteme otomatik kayıt (AutoLogger) gibi özelleştirilmiş oturumlar oluşturmak da mümkündür. Her oturumun güvenlik tanımlayıcıları ve yapılandırması kayıt defterinde depolanır
2. **GUID ve Manifestler**: ETW provider'ları, manifest dosyalarında tanımlanan GUID'lerle ayırt edilir . Aynı şekilde her görev (task), opcode ve kanal benzersiz ID'lere sahiptir. ETW manifestleri XML tabanlıdır ve olay verilerinin şematik formatını, simgelerini, mesaj şablonlarını içerir. Manuel manifest derlemeyle (mc.exe) *.bin ve header dosyaları oluşturulur, böylece uygulamalar statik veya dinamik olarak olay üretebilir
3. **ETL Dosyaları**: ETW verileri genellikle .etl uzantılı ikili günlük dosyalarına yazılır. Bu dosyalar kernel bufferlarından alınan ham olayı içerir ve standart formlar içermez; çözümlemek için Windows Event Viewer, tracerpt.exe veya Windows Performance Analyzer gibi araçlar gerekir. ETW olayları zengin meta-veri içerir (zaman damgası, iş parçacığı/process kimliği, CPU bilgisi vb.) ve depolandıktan sonra parse edilerek analiz edilir
4. **Özel Oturumlar ve Kanallar**: ETW'de kanal (channel) kavramı da vardır; her provider bir veya daha fazla kanala bağlı olabilir (Admin, Operational, Analytic vb.). Örneğin PowerShell ve AMSI gibi sistem bileşenleri, kendilerine ait Operational kanallara sahiptir. Ancak bazı kanallar, varsayılan izleme oturumlarına dahil olmadığından, özel izleme oturumları açılması gerekebilir. Bu yapılarda Registry'de HKLM\SYSTEM\CurrentControlSet\Control\WMI\Autologger altında ayarlar tutulur

IBM'in ETW Tehdit Modeli



REF: https://www.ibm.com/think/x-force/direct-kernel-object-manipulation-attacks-etw-providers

Yukarıdaki tehtit modelinde saldırganların user mode ve Kernel modda müdahale edebildikleri bileşenler gösterilmektedir.

ETW Sağlayıcıları (Providers): Sistem ve uygulama olaylarını kaydeden bileşenler. Örn PowerShell, AMSI, Windows Kernel bileşenleri.

Oturumlar (Sessions): ETW sağlayıcılarının gönderdiği olayların toplandığı ve saklandığı yapılar. Örn: Default, Kernel-Logger vb.

ETW Kanalları (Channels): Olayların türüne göre ayrıldığı yollar.

SALDIRI ÇEŞİTLERİ

ETW Tampering

1) **Uygulama seviyesi ETW API'lerini yama (patch) etmek:**

User modundaki uygulamalar veya sistem bileşenleri, ETW sağlayıcılarına olay yazmak için belirli API'leri çağırır. Bu API'lerin en kritik olanlarından bazıları **ntdll.dll** içindeki "**EtwEventWrite** ,**EtwEventWriteFull** ,**EtwEventWriteTransfer**" fonksiyonlardır. Saldırganlar bu fonksiyonları bellek üzerinde patch'leyerek (hooking veya inline patching) ETW iletim işlevlerine müdahale edebilir.

En sık görülen yöntemlerden biri, bu API'lerin başlangıcına küçük ama etkili işlemlerin eklenmesidir. Bu işlevlerin başlangıcına bir **RET** veya **XOR EAX,EAX; RET** gibi komutlar ekleyerek çağrıların hemen işlemleri gerçekleştirmeden sonlanması yada "failure" döndürmesi sağlanır. Böylece ilgili olayların kaydedilmesi engellenir. Bu teknik sayesinde, uygulamanın veya zararlı yazılımın gerçekleştirdiği aktiviteler Windows log mekanizmasına hiç ulaşmaz, dolayısıyla EDR / SIEM / AV gibi güvenlik ürünleri için görünmez olur.

ÖRNEK POC: Unhooking Event Tracing for Windows < BorderGate

2) **Syscall çağrısını devre dışı bırakmak:**

Windows'ta ETW event'ları yalnızca üst seviye API'lerle iletilmez. Bu eventlar en sonunda Kernel geçiş yaparak "**syscall**" fonksiyonlarını tetikler (ör. **NtTraceEvent**, **ZwTraceEvent**). Kullanılan Bu fonksiyonlar "**ntdll.dll**" içinde bulunur ve kullanıcı modundan çekirdeğe bir sistem çağrısı (syscall) yaparak olayların ETW altyapısına kaydedilmesini sağlar.

Bir saldırgan bu syscall'lara müdahale ederek ilgili logların Kernel'e geçişini engeller. Böylelikle ilglili loglar ETW altyapısına ulaşamaz.Bu geçiş, sisteme gömülü çağrı stub'larının ilk komutunu RET ile değiştirerek engellenebilir.

Bu saldırıdaki Dikkat edilmesi gereken nokta;
EDR ürünleri çoğunlukla **EtwEventWrite** gibi API'leri izler. Ancak saldırgan doğrudan **NtTraceEvent'i** patch ederse, bu kontroller atlanır. Çünkü çoğu güvenlik ürünü syscall'larını sürekli denetlemez.

PoC and Ref: Bypassing ETW For Fun and Profit | White Knight Labs

3) **<u>Obfuscation, Encrypted or Stealth SYSCALL</u>:**

Normal şartlarda bir process bir API çağrısı yaptığında, **call stack** izlenerek bu çağrının hangi fonksiyonlardan geçtiği görülebilir. Ancak APT grupları, call stack üzerinde sahte katmanlar oluşturarak veya orijinal çağrı akışını gizleyerek güvenlik ürünlerinden kaçınma teknikleri uygular.

Bu bağlamda öne çıkan yöntemlerden biri "**stealth syscall**" tekniğidir. Bu yaklaşımda çekirdek çağrıları (**syscall**) doğrudan, encrypted veya obfuscated olarak çalıştırılır. Amaç, güvenlik araçlarının ham syscall çağrısını tanımasını ya da takip etmesini engellemektir.

Son raporlara göre bu teknikler genellikle üç ana yöntemin birleşiminden oluşur:

- Call stack forgery
- ETW API hook bypass
- Şifreli veya gizlenmiş syscall yürütme

Bu kombinasyon, geleneksel algılama mekanizmalarını boşa düşürmekte ve özellikle kullanıcı modundaki ETW yama tespitlerini etkisiz hale getirmektedir.

Çevresel/Ortamsal (Env/Registry) Saldırılar

1) **ETW Günlük Dosyalarını Manipüle etmek yada silmek:**

Saldırganlar, ETW izleme kayıtlarının yazıldığı dosya ve klasörleri hedef alabilir. Örneğin **Slingshot APT** saldırılarında, ETW dosyalarının uzantılarını "**.tmp**" uzantısıyla değiştirerek sisteme iz bırakmaktan kaçınmıştır. Benzer şekilde, logların default konumlarından başka bir konuma taşınması ya da silinmesi yöntemleriyle ETW kayıtları gizlenebilir.

2) **Windows Yönetim Altyapısı (WMI) ve Logger Ayarlarına Müdahale etmek:**

Windows'ta ETW oturumlarını kontrol eden çevresel ayarlar ve servisler üzerinde değişiklik yapılarak izleme durdurulabilir.

Örneğin LockerGoga, "**wevtutil"** kullanarak **Microsoft-Windows-WMI-Activity Provider**'ını kapatmıştır. Bu sayede WMI tabanlı "process creation" gibi aktiviteler loglanmamıştır.

Ayrıca Bir zararlı bir driver yükleyerek ETW oturumlarının konfigürasyonu ve maksimum Provider sayısı gibi **WMI_LOGGER_CONTEXT** yapıları değiştirilebilir. Saldırganlar Bu değerleri degiştirerik oluşan yeni aktivitelerin loglanmasının önler.

3) **Sistem Sürücülerindeki Korumaları İptal Etme:**

Bazı önemli ETW providerları kendileri gibi **Protected Process Light** (PPL) seviyesinde calışır. Saldırganlar, PPL korumasını geçici olarak kaldırıp kritik Provider(Statü farkı oluşturarak) devre dışı bırakabilir .

Örneğin, PPL'ye sahip bir işleme sinyaller gönderilerek veya imzasız bir sürücü ile PPL pasif hale getirilerek Korumalı ETW sağlayıcılarının kayıtları durdurulabilir. Böylece sistem servisleri tarafından oluşturulan bazı olay kayıtları engellenebilir.

User Mode ETW Sağlayıcılarına Yönelik Saldırılar

1) **Provider Bazlı Hooklama:**

Saldırgan, hedef uygulamanın bellek alanına girerek (DLL injection, code injection) bu sağlayıcıların işlevlerine (ör. **EventWrite** , **TraceLoggingWrite** vb.) müdahale edebilir.

Örneğin, Saldırgan bir .NET uygulamasındaki **EventWrite** veya **TraceLoggingWrite** gibi fonksiyonları hook'lar yada patch'ler. Böylelikle bu fonksiyonlar cağrılsa da sonuclar ETW'ye gitmez, ya da boş bir fonksiyona yönlendirilebilir.

2) **ETW Sağlayıcı Tetikleyicilerini Kaldırma:**

User Moda çalışan bir Providerı devre dışı bırakmak için ilgili işleme enjekte olup işlem içinde ilgili GUID'e karşılık gelen etkinliği pasif hale getirebilir.

Örneğin **EnableTraceEx/EnableTraceEx2** çağrısını kullanarak belirli bir sağlayıcıyı devre dışı bırakmak mümkündür. Bu tür işlemler sırasında EDR'ler genellikle Default olarak işlem belleğini tarayıp anomaliyi bildirir.

Çekirdek Modu ETW Sağlayıcılarına Yönelik Saldırılar

1) **WPP/Kernel Trace Hook'ları:**

Windows Performance Tracing (WPP) Windows sürücüleri ve kernel bileşenleri için kullanılan bir izleme mekanizmasıdır. Kernel ETW sağlayıcıları olaylarını "**TraceLoggingRegister"** veya "**EtwRegister"** üzerinden ETW'ye kaydettirir. Normalde kullanıcı modundaki ETW provider'ları "**EventWrite"** ile olay gönderirken, sürücü seviyesinde bu işlevler kernel tarafından yönetilir.

Saldırganlar, Kendi Driver'larını yükleyerek bu sağlayıcılar üzerinde değişiklik yapabilir. Örneğin, "**TraceLoggingRegister"** yoluyla kaydedilmiş kernel olaylarına müdahale edilebilir ya da "**EtwRegister"** nesneleri patch'lenebilir. Bu sayede Provider ve Sessionlar acık görünsede işlem olayları veya sürücü aktiviteleri ETW ile raporlanmaz.

2) **Protected (Manifest Tabanlı) Kernel Provider'larının Devre Dışı Bırakılması:**

Windows çekirdeği bazı kritik olayları (ör. kimlik doğrulama, token işlemleri, thread/process aktiviteleri) yalnızca belirli yetkilerle (örneğin PPL yetkisiyle) normal güvenlik logları dışında ek olarak kernel ETW provider'ları tarafından da izlenir. Bu sağlayıcılar "**manifest-based**" olup gizli kabul edilir çünkü yalnızca özel yetkilere sahip süreçler tarafından kullanılabilir.

Erişim genellikle **Protected Process Light** (PPL) statüsüne sahip işlemler ile sınırlıdır. Saldırgan Kendi driverı ile PPL'yi kaldırıp çekirdek içi izleme sağlayıcılarını engelleyerek "**Protected**" çekirdek olaylarının toplanmasını önleyebilir.

3) **Cekirdek ETW Tablolarının DKOM ile Değiştirilmesi:**

ETW çekirdek tarafında yapısal tablolar ile yönetilir

- **WMI_LOGGER_CONTEXT** → aktif oturumların kontrolü
- **ETW_GUID_ENTRY** → provider'ların kayıtları
- **TRACE_ENABLE_INFO** → hangi event'in aktif/pasif olacağı
- **TraceInformation tabl**osu → Windows 10/11'de kernel ETW işlemlerinin durumu

Bu tablolar kernel memory'de tutulur ve normalde kullanıcı modundan erişilemez.

Eğer saldırgan Kernel seviyesinde kod çalıştırma yetkisine sahipse, **WMI_LOGGER_CONTEXT** veya diğer ETW iç yapıları DKOM **(Direct Kernel Object Manipulation**) ile değiştirilebilir.

Örneğin **TraceInformation tablosu** overwrite yada "**NULL"** set edilerek tüm Kernel ETW işlemlerine müdahale edilebilir

ETW Güvenligi Savunma Stratejileri ve önerileri

1) Dosya ve Kayıt Manipülasyonlarına Karşı Savunma

- **File Integrity Monitoring (FIM)** → .etl, .tmp, %SystemRoot%\System32\winevt\Logs gibi kritik ETW dosyalarının bütünlük takibi yapılmalı.
- **Sysmon + EDR kuralları** → ETW dosyalarının rename, delete, move edilmesi gibi anormal işlemler izlenmeli.
- **Disk Denetimi** → "Slingshot" tarzı rename saldırılarında, ETW dosyaları dışında beklenmeyen anormal dosya tür (örn .tmp) yoğunluğu izlenmeli.

2) WMI ve Logger Context Manipülasyonuna Karşı Savunma

- **WMI Event Subscription Monitoring** → WMI sağlayıcılarının devre dışı bırakılması / config değişiklikleri için sürekli izleme.
- **ETW Internal Checks** → WMI_LOGGER_CONTEXT memory validation ve session integrity sürekli kontrol edilmeli.
- **Event ID 23-24** → Event loglar yada sysmon ile WMI activity loglanabilir, suspicious stop/start işlemleri tetikleyici olabilir.

3) Protected Process Light (PPL) Bypass'a Karşı Savunma

- **Driver Signing Enforcement** → İmzasız sürücülere kesinlikle izin verilmemeli (HVCI + Device Guard).
- **Kernel Debug Mode Off** → Kernel debug açıkken saldırganlar PPL atlatmayı kolay yapabiliyor.
- **ETW Heartbeat** → Kritik ETW sağlayıcılarının çalışıp çalışmadığını doğrulayan periyodik testler (ör. LSASS gibi kritik protected ETW provider'larından beklenen event gelmezse alarm!!).

4) Kullanıcı Modu Hooklama ve GUID Manipülasyonuna Karşı Savunma

- **EDR Process Memory Scans** → EventWrite, TraceLoggingWrite gibi API'lerin patchlenip patchlenmediği Kontrol edilmeli.
- **ETW API Call Heuristics** → EnableTraceEx2 gibi çağrılar normalde sistem süreçleri dışında çok nadir yapılır. suspicious activity!
- **.NET / CLR Monitoring** → Managed uygulamalarda ETW hook girişimleri için AppDomain içi anormallikler izlenebilir.

5) Kernel Trace Hook ve DKOM Saldırılarına Karşı Savunma

- **Kernel ETW Watchdogs** → ETW tablosunun (TraceInformation, EtwGuidEntry vs.) runtime integrity checkleri izlenebilir
- **Hypervisor-Based Protection (HVCI, VBS)** → DKOM atlatılamaz hale gelir. Memory guard'lar kullanılabilir.
- **ETW Cross-Validation** → Kernel'den gelen olaylar ile User-mode ETW loglarının tutarlılık kontrolü yapılabilir.

6) Genel Strateji

- **Redundancy Logging** → ETW tek kaynağa bırakılmamalı. Sysmon, EDR driver telemetry, Network telemetry vs. ile korelasyon sağlanmalı.
- **Decoy Providers** → Honeypot mantığında sahte ETW provider eklenerek saldırganın manipülasyon girişimleri yakalanabilir.
- **Anomaly Detection (ML tabanlı)** → Normal ETW event rate düşüşleri izlenebilir.
- **Kritik süreçleri koru** → PPL ve WDAC koruması kritik sağlayıcıların güvenliğini artırır.
- **Çok katmanlı savunma uygula**: Kernel + API integrity + dosya/log izleme + telemetri korelasyonu

Önerilen Birleşik Strateji

1. **Kernel ETW + PPL / VBS / HVCI** → Temel savunma hattı, anti-tamper.
2. **ETW API & Session Integrity** → Kullanıcı ve kernel modunda manipülasyon tespiti.
3. **Dosya / Log Monitoring** → FIM + Sysmon ile anormal file event'leri yakalama.
4. **Sürücü Güvenliği** → Signed drivers, Secure Boot, patch management.
5. **Anomali ve Redundancy** → ML tabanlı event monitoring, cross-log validation, decoy providers
6. **Çok Katmanlı Savunma** → Tek bir önlem yerine kombinasyon ile saldırı tespiti ve önleme.

Saldırı Türü	Teknik Açıklama	Ön Koşullar / Yetki	Etki Düzeyi	Savunma / Öneriler
Kullanıcı Modu ETW API Patch	EtwEventWrite, EventWriteEx vb. fonksiyonların pachlenmesi.	User-mode, genelde Admin hakları gerekli değil.	Sadece hedef süreçten çıkan logları susturur.	API bütünlük denetimi, EDR hook doğrulaması, provider heartbeat, memory scan, telemetri sağlık testi, cross-log korelasyonu.
Syscall Gövdesini Devre Dışı Bırakmak	NtTraceEvent gibi fonksiyonların gövdesi boşaltılır, stealth syscall veya stack trace gizleme.	Kernel-mode erişim gerekir.	Sistem genelinde ilgili syscall loglaması devre dışı.	HVCI, PatchGuard, kernel memory integrity monitoring, stack trace analizi, davranış profilleme
Obfuscation / Stealth Syscall	Syscall tablosuna gizli/şifreli erişim; stack trace gizleme.	User-mode ama ileri düzey exploit / EDR bypass gerekir.	Süreç bazlı görünmezlik, olayların loga düşmemesi.	Stack trace analizi, davranış profilleme.
ETW Günlük Dosyalarını Silmek / Manipüle Etmek	.etl / .evtx log dosyalarının silinmesi, yeniden adlandırılması veya başka konuma taşınması.	Admin yetkis gerekir.	Mevcut kayıtlar silinir, taşınır yada adresleri degiştirilir.	File integrity monitoring, Sysmon ile dosya silme izleme ve anomaly detection.
WMI / Logger Ayarlarına Müdahale	HKLM\SYSTEM\...\Autologger veya provider config devre dışı bırakılır.	Admin / SYSTEM Yetkisi gerekir.	Tüm sistemde belirli kanallar log üretmez hale gelir.	WMI session integrity, registry ve dosya monitoring, telemetri health check
Sistem Sürücülerindeki Korumaları İptal Etme (BYOVD)	Zafiyetli sürücü yükleyerek ETW/EDR callback'leri kaldırma. (PPL bypass)	Admin yetkisi + sürücü yükleme (Ring-0).	Tüm sistemde EDR/ETW tamamen susturulabilir.	Microsoft driver blocklist, Secure Boot, HVCI, PatchGuard, WDAC, VBS, callback monitoring (PsSetLoadImageNotifyRoutine).
Sağlayıcı Bazlı Hooklama	Belirli ETW GUID sağlayıcılarının seçici devre dışı bırakılması; EventWrite veya TraceLoggingWrite hook.	Kernel veya uygulamaya göre user mode erişimi gerekir.	Yalnızca seçilen kritik sağlayıcılar susturulur (örn. Defender, AMSI).	Sağlayıcı bütünlük testi, API integrity check, EDR memory monitoring.
ETW Sağlayıcı Tetikleyicilerini Kaldırma	EtwpActiveSystemLoggers veya context pointer'ların NULL'a çekilmesi.	Kernel erişimi (DKOM).	Tüm aktif oturumlar sessizce susturulur.	Bellek bütünlüğü izleme, kernel anti-tamper, DKOM detection.
WPP / Kernel Trace Hook'ları	Kernel trace makrolarına hook atarak çıktıyı manipüle etme. TraceLoggingRegister / EtwRegister patch.	Kernel erişimi gerekir.	Kernel debug/trace altyapısı manipüle edilir.	Kernel memory diff, trace provider health check, Driver signing enforcement, HVCI/VBS, callback monitoring.
Protected Kernel Sağlayıcılarını Devre Dışı Bırakma	Manifest tabanlı kritik sağlayıcıların loglaması engellenir. PPL bypass ile gizli ETW provider durdurulur.	Kernel erişimi gerekir.	Kritik sistem loglaması (örn. process creation) devre dışı.	Provider health test, ETW session integrity kontrolü, PatchGuard, PPL state monitoring, VBS, WDAC, Secure Boot.
Çekirdek ETW Tablolarını DKOM ile Değiştirmek	_ETW_REG_ENTRY, _ETW_GUID_ENTRY gibi yapılar kernel bellekte DKOM ile değiştirilir, TraceInformation tablosuna yazılır.	Kernel erişimi (Ring-0).	Tüm ETW altyapısı bozulur, sistem global olarak kör olur.	Kernel memory forensic, Secure Kernel (VBS/Isolated User Mode), anti-DKOM araçları, decoy providers, cross-logging.

Referanslar

From Cobalt Strike to Mimikatz: A Deep Dive into the SLOW#TEMPEST Campaign Targeting Chinese Users - Securonix

Tampering with Windows Event Tracing: Background, Offense, and Defense | by Palantir | Palantir Blog

New ETW Attacks Can Allow Hackers to 'Blind' Security Products - SecurityWeek

Remcos RAT Targets Europe: New AMSI and ETW Evasion Tactics Uncovered

Design issues of modern EDRs: bypassing ETW-based solutions

ETW Forensics - Why use Event Tracing for Windows over EventLog? - - JPCERT/CC Eyes | JPCERT Coordination Center official Blog

Instrumenting Your Code with ETW | Microsoft Learn

LockBit Ransomware Side-loads Cobalt Strike Beacon with Legitimate VMware Utility | SentinelOne

Unhooking Event Tracing for Windows < BorderGate

Anti-Tampering and Detection Evasion – Bitdefender TechZone

Bypassing ETW For Fun and Profit | White Knight Labs

Kernel ETW is the best ETW — Elastic Security Labs